Freight Rate Prediction

Validation approach, model and the fixed December forecast

Item	Detail
Assessment	Spotter - Machine Learning Engineer
Candidate	Pranto Bosu
Repository	https://github.com/bosu0077/Freight-rate-prediction-project.git
Notebook	Freight_Rate_Prediction.ipynb (runs end-to-end, then calls score.py)
Date	27 October 2026

1. Summary

The task is a **forecast**: all 48,000 labelled loads are from January-October 2025, while the 12,000 loads to predict are from November-December 2025. Every model-selection decision was therefore made on future months, never on a random split.

- **Validation design:** time-based. Develop on Jan-Aug, score once on a Sep-Oct holdout (a 2-month-ahead horizon, like Nov-Dec), tune on expanding-window folds inside Jan-Aug, then refit on all of Jan-Oct.
- **Final model:** a two-stage pipeline - gradient-boosted lane price (distance, geography, equipment, weight) multiplied by a linear market level (trend, daily market index, quarter-end surge).
- **Holdout accuracy (Sep-Oct):** MAPE 1.68%, MAE $38.44, R^2 0.9984; 97.9% of loads within ±5%.
- **Why the split matters:** under random 5-fold CV the model that adds the raw quote_signal and market_index looks best (1.22% vs 1.53% MAPE for the final model) and a naive all-columns model looks about as good (1.56%). On future months they reach 1.95% and 3.52%, while the final model holds at 1.70%.
- **December chart:** $818.38-$883.59 for the fixed Lexington-Fort Wayne Dry Van load, with a mid-week peak, a Sunday-Monday dip and a ~4.0% quarter-end surge into 31 December.

2. Key findings from the data

- **Pricing structure:** rate scales with distance while the rate per mile tapers for long hauls; Reefer > Flatbed > Dry Van.
- **Corrupted labels:** 677 loads (1.41%) are 2-6× above or below what identical loads cost, separated from normal loads by an empty gap in the residuals - most likely keying or unit errors.
- **quote_signal is unstable:** it equals the rate per mile in Jan-Mar, Jun and Sep ($r \approx +0.99$), is its mirror image in Apr, May, Jul and Oct ($r \approx -0.99$) and is noise in August. A label-free check shows it is noise in Nov-Dec, so it is excluded.
- **market_index drifts:** at load level it scatters by about 2.5% (one standard deviation) around a daily market level; that level co-moves with rates in Jan-Aug, but in Sep-Oct the index falls back to its January level while rates stay high. It is used only as a daily average with a small linear elasticity next to a trend.
- **Quarter-end surge:** rates climb ~4% over the last ~25 days of every quarter (March, June, September) - December is a quarter end.
- **Unseen cities:** 8 cities (1,447 validation loads) never appear in training, so the model uses coordinates, not city IDs.

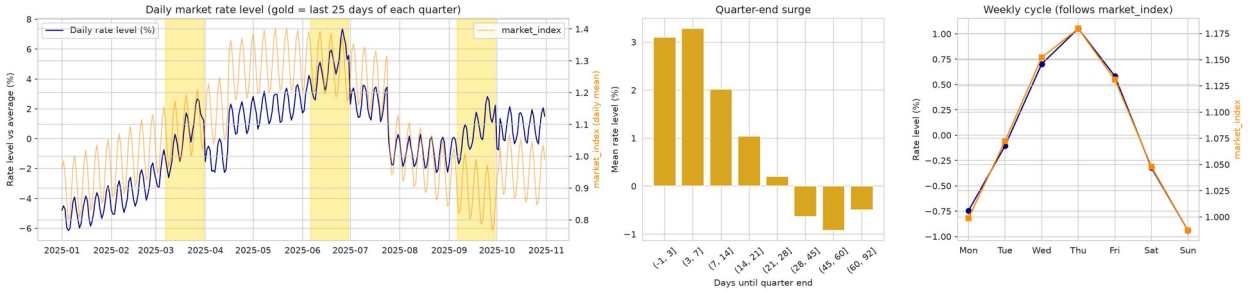


Figure 1 - Daily market rate level (lane effects removed) vs market_index, quarter-end surge and weekly cycle.

3. Data-quality issues and treatment

Issue	Dev rows	Validation rows	Treatment
Corrupted posted_rate labels (2-6x off)	677	-	Removed from training & clean evaluation
Negative weight (sign error)	292	145	Absolute value (pipeline)
Missing weight	300	165	Median weight of the same equipment (learned on training folds)
Missing market_index	374	249	Not needed: daily market index used
market_index load-level noise / drift	all	all	Daily average + linear elasticity with trend
quote_signal regime flips by month	all	all	Excluded from the model
Cities only in validation	-	1,447	Coordinates instead of city IDs
Weight cap 47,500 / floor 5,000; distance floor 70 mi	1,204	-	Valid business limits - kept
December inputs lack coordinates / market data	-	31 rows	Filled from the lookup tables

All fixes except the label removal run inside the scikit-learn pipeline, so the holdout, validation and December rows receive exactly the same treatment. The two lookups (city coordinates, daily market index) are built from features only - never from labels.

4. Validation and train/test split approach

4.1 Principle: validate the way the model will be used

The model has to price loads two months beyond the last label. A random train/test split would mix every month into both sides, so it would reward models that memorise month-specific behaviour (the quote_signal regimes, the market level) and would hide the drift. The split therefore follows time.

4.2 Split design

Set	Period	Rows	Used for
Development	Jan 1 - Aug 31, 2025	37,944	Fitting, model comparison, tuning
Tuning fold 1	train Jan-Apr / validate May-Jun	-	RandomizedSearchCV (MAE)
Tuning fold 2	train Jan-Jun / validate Jul-Aug	-	RandomizedSearchCV (MAE)
Holdout	Sep 1 - Oct 31, 2025	9,379	One final, unbiased estimate
Final training	Jan 1 - Oct 31, 2025	47,323	Refit before predicting Nov-Dec
Validation (to predict)	Nov 1 - Dec 31, 2025	12,000	Submitted predictions

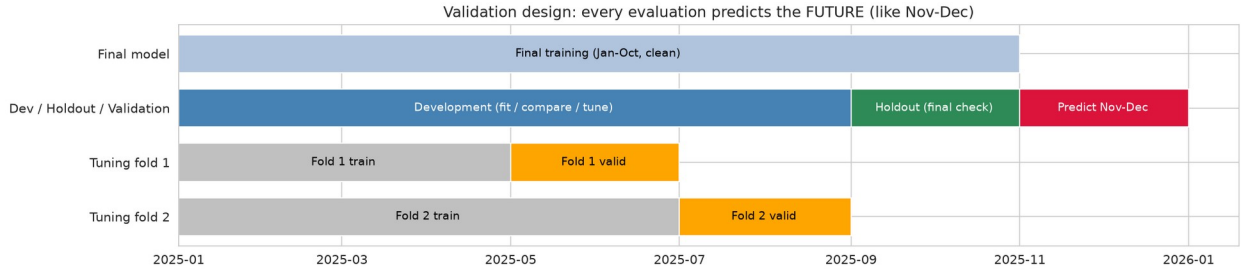


Figure 2 - Every evaluation trains on the past and predicts the future, with a 2-month horizon like Nov-Dec.

4.3 Evidence: random CV would have chosen the wrong model

Three model designs were scored twice: with random 5-fold CV inside Jan-Aug, and on the Sep-Oct holdout after training on Jan-Aug.

Model	Random 5-fold CV MAPE	Time holdout MAPE	Holdout / CV
Naive HistGB (all raw columns)	1.56% (rank 3)	3.52% (rank 3)	2.26×
HistGB + raw quote & market signals	1.22% (rank 1)	1.95% (rank 2)	1.59×
Two-stage production model	**1.53% (rank 2)**	**1.70% (rank 1)**	**1.11×**

Random CV ranks the raw-signal model first and puts the naive model level with the final model, because with random folds every month sits on both sides of the split and a tree can learn each month's quote_signal regime and market level from neighbouring days. On future months the final model is best and the naive model's error more than doubles. All tuning therefore uses time-series folds, and the Sep-Oct holdout is the headline number.

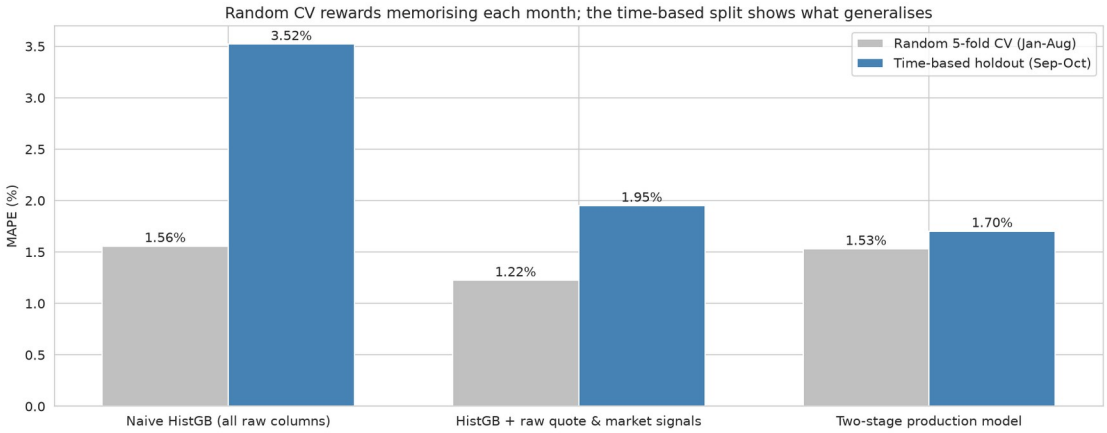


Figure 3 - Random CV (grey) vs the time-based holdout (blue) for the three model designs.

4.4 Label quality and leakage controls

- **Corrupted labels:** flagged once with a robust (absolute-error) gradient-boosting model using 5-fold out-of-fold predictions; |log residual| > 0.5 means the rate is >1.65× or <0.61× the expected rate. They are removed from training.
- **Two holdout scores:** clean labels: MAPE 1.68%, MAE $38.44; including the flagged labels: MAPE 4.20%, MAE $94.49 - the gap is the cost of labels that are wrong, not of the model.
- **Pipeline-fitted preprocessing:** imputation values and every model parameter are learned inside each training fold only (scikit-learn Pipeline + custom transformer).
- **Label-free drift check:** before predicting Nov-Dec, quote_signal was correlated with the model's predicted rate per mile on the unlabelled validation loads: r = -0.001 in November and -0.013 in December, versus |r| = 0.98-0.99 in the direct and inverted training months. The signal is noise in Nov-Dec, which confirms the exclusion.
- **Unseen-city stress test:** removing 8 cities from training gives 1.94% MAPE on loads touching them vs 1.69% for known cities.

5. Model choice and results

log(rate) = **lane price** (HistGradientBoosting on distance, coordinates, circuity, equipment, weight) + **market level** (linear in day index, log daily market index and a quarter-end ramp). The two parts are fitted jointly by back-fitting. Trees capture the non-linear lane surface; the linear market level extrapolates the trend into Nov-Dec, keeps a stable market elasticity and applies the quarter-end surge to December - things a single tree model cannot do.

Model (train Jan-Aug, score Sep-Oct)	MAE ($)	MAPE (%)	R^2
Two-stage production model	**38.90**	**1.70**	**0.9983**
HistGradientBoosting (log)	44.07	1.89	0.9979
HistGB + raw quote & market signals	46.84	1.95	0.9976
Gradient Boosting (log)	46.37	1.97	0.9977
Ridge (log, alpha=1)	47.62	2.10	0.9976
Linear Regression (log)	47.62	2.10	0.9976
Random Forest (log)	62.70	2.62	0.9956
Decision Tree (log)	77.02	3.37	0.9936

Model (train Jan-Aug, score Sep-Oct)	MAE ($)	MAPE (%)	R^2
Naive HistGB (all raw columns)	84.88	3.52	0.9933
Rate-per-mile heuristic	83.39	3.74	0.9929

Tuned configuration (time-series RandomizedSearchCV): min_samples_leaf=10, max_leaf_nodes=63, learning_rate=0.03, l2_regularization=0, ramp_days=30. Learned market effects on Jan-Oct: trend 0.59% per 30 days, market-index elasticity 0.153, quarter-end surge 3.97% on the last day of the quarter.

6. Fixed December prediction chart (score.py)

The chart below was produced by the provided score.py after it validated both output files ("Validated 12,000 final predictions. Validated 31 fixed December predictions."). Inputs are fixed - Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb - and only the date changes. Coordinates were filled from the city lookup and the market index from each December date's average across the validation loads.

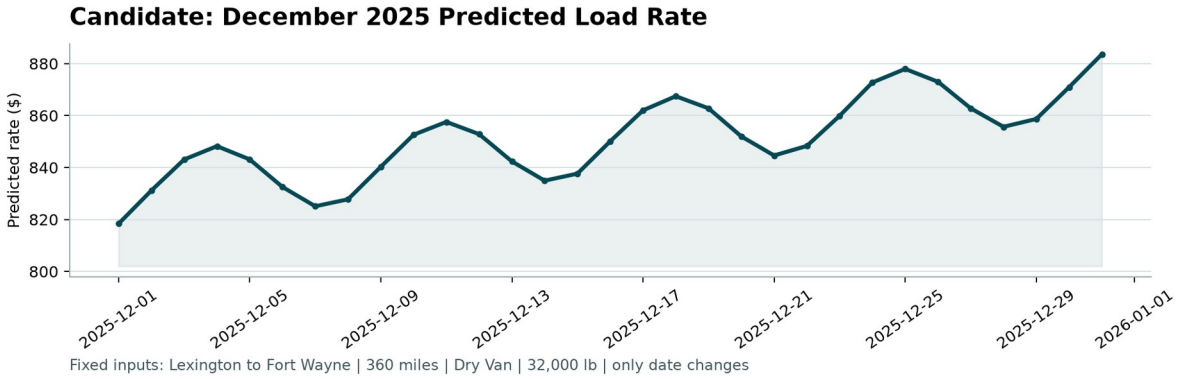


Figure 4 - candidate_december.png, generated by score.py.

- **Weekly cycle:** the daily market index is highest on Wednesday-Thursday and lowest on Sunday-Monday, so the rate peaks mid-week and dips around the weekend (elasticity 0.15).
- **Quarter-end surge:** the tuned ramp covers the last 30 days of the quarter, so from 2 December the rate climbs steadily and is about 4.0% higher on 31 December, as it was before the end of March, June and September.
- **Level check:** $818.38-$883.59 sits inside the lane's historical Dry Van range (21 loads, $757.93-$934.37, median $807.89), slightly above the median because of the trend and the quarter-end surge.
- **Not modelled:** holiday effects (e.g. Christmas week) cannot be learned from January-October data.
- **Known limitation:** when rates and market_index decouple, the market level adapts with a lag (holdout mean signed error -0.26% in September, -0.97% in October); the final refit on Jan-Oct already includes the Sep-Oct level.

7. Reproducibility

```
python -m pip install -r requirements.txt
jupyter nbconvert --to notebook --execute --inplace Freight_Rate_Prediction.ipynb
python score.py --predictions validation_predictions.csv --december-predictions
data/december_chart_inputs.csv
```

Fixed random seed 42. Tested with Python 3.9 (scikit-learn 1.0.2 and 1.6) and Python 3.12 (scikit-learn 1.9): every version selects the same model and gives the same holdout MAPE to two decimals.